

Digital Design

Chapter 3:
Sequential Logic Design -- Controllers

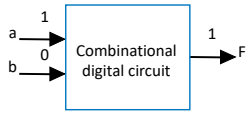
Slides to accompany the textbook Digital Design, with RTL Design, VHDL, and Verilog, 2nd Edition,
by Frank Vahid, John Wiley and Sons Publishers, 2010.
http://www.ddvahid.com

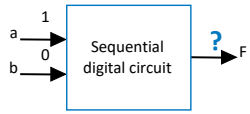
Copyright © 2010 Frank Vahid

*Instructors of courses requiring Vahid's Digital Design textbook (published by John Wiley and Sons) have permission to modify and use these slides for customary course-related activities, subject to keeping this copyright notice in place and unmodified. These slides may be posted as unanimated pdf versions on publicly-accessible course websites.. PowerPoint source (or pdf with animations) may **not** be posted to publicly-accessible websites, but may be posted for students on internal protected sites or distributed directly to students by other electronic means. Instructors may make printouts of the slides available to students for a reasonable photocopying charge, without incurring royalties. Any other use requires explicit permission. Instructors may obtain PowerPoint source or obtain special use permissions from Wiley – see http://www.ddvahid.com for information.*

Introduction

- Sequential circuit
  - Output depends not just on present inputs (as in combinational circuit), but on past sequence of inputs
    - Stores bits, also known as having "state"
  - Simple example: a circuit that counts up in binary
- In this chapter, we will:
  - Design a new building block, a **flip-flop**, that stores one bit
  - Combine that block to build multi-bit storage – a **register**
  - Describe the sequential behavior using a **finite state machine**
  - Convert a finite state machine to a **controller** – a sequential circuit having a register and combinational logic

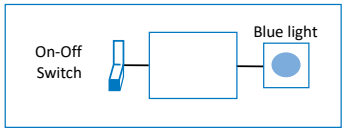




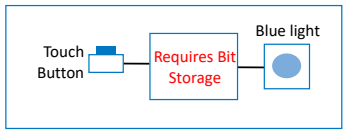
Must know sequence of past inputs to know output

Note: Slides with animation are denoted with a small red "a" near the animated items

Example Needing Bit Storage



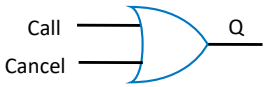
*Button **DOWN** – light turns on and stays on*
*Button **UP** – light turns off and stays off*



(Initially light is off)
*Button **TOUCHED** – light turns on and stays on*
*Button **TOUCHED** – light turns off and stays off*

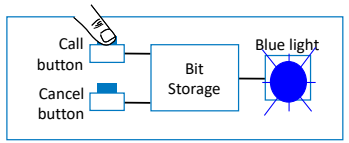
Example Needing Bit Storage

- Flight attendant call button
  - Press call: light turns on
    - ***Stays on*** after button released
  - Press cancel: light turns off
  - Logic gate circuit to implement this?

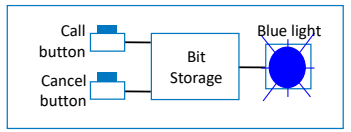


Doesn't work. Q=1 when Call=1, but doesn't stay 1 when Call returns to 0

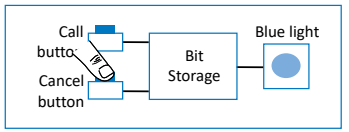
Need some form of "feedback" in the circuit



1. Call button pressed – light turns on



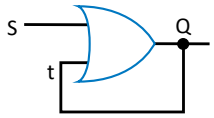
*2. Call button released – light **stays on***

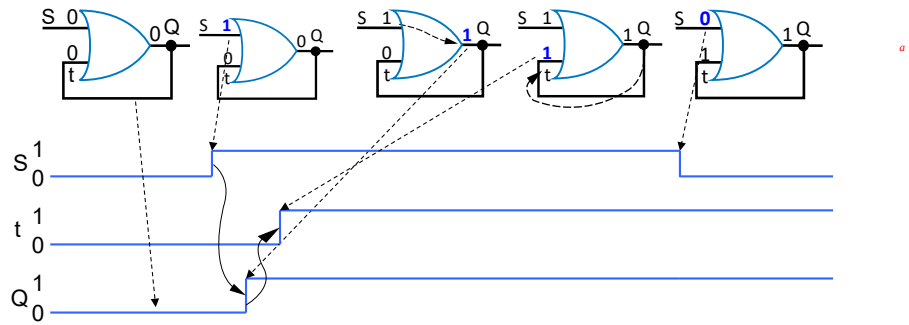


3. Cancel button pressed – light turns off

First attempt at Bit Storage

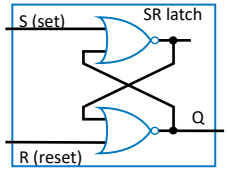
- We need some sort of feedback
  - Does circuit on the right do what we want?
    - No: Once Q becomes 1 (when S=1), Q stays 1 forever – no value of S can bring Q back to 0

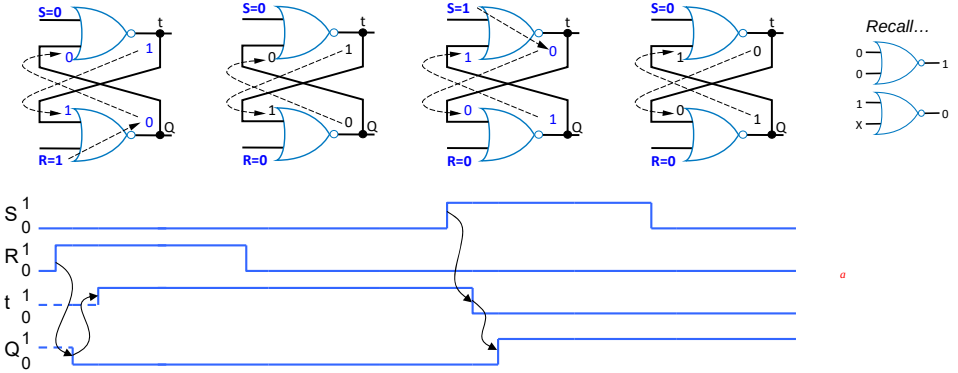




Bit Storage Using an SR Latch

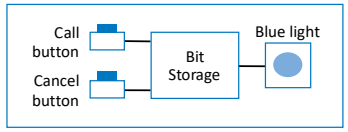
- Does the circuit to the right, with cross-coupled NOR gates, do what we want?
  - Yes! How did someone come up with that circuit? Maybe just trial and error, a bit of insight...

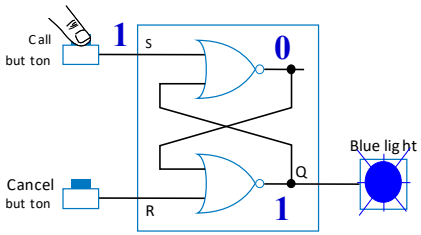




Example Using SR Latch for Bit Storage

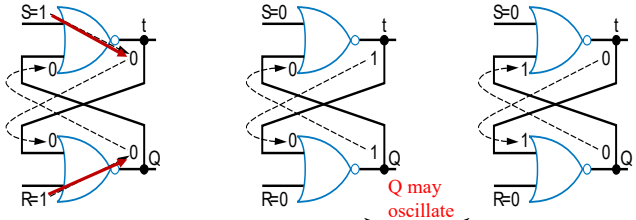
- SR latch can serve as bit storage in previous example of **flight-attendant call button**
  - Call=1 : sets Q to 1
    - Q stays 1 even after Call=0
  - Cancel=1 : resets Q to 0
- But, there's a problem...

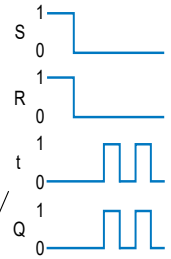




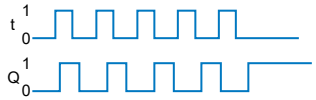
Problem with SR Latch when SR=11
The Race Condition

- Problem
  - If S=1 and R=1 simultaneously, we don't know what value Q will take when S=0 and R=0 made simultaneously.





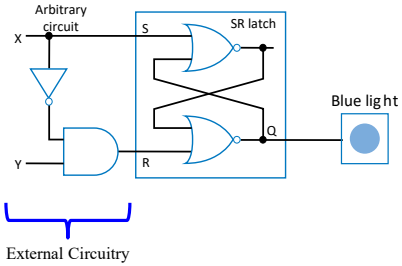
Q may oscillate. Then, because one path will be slightly longer than the other, Q will eventually settle to 1 or 0 – but we don't know which.



A situation in which the final output of a sequential circuit depends on the delays of gates and wires is a ***race condition***.

Problem with SR Latch — 1: User ➔ Oscillation

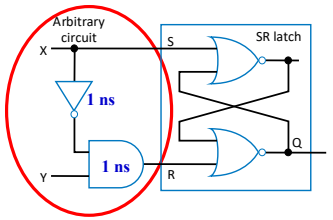
- Problem occurs when a user presses two buttons at same time
- **Solution:** Designer might try to avoid problem using external circuit
  - External circuitry such as given in the figure avoids such situations



However, SR=11 **can still occur** on Basic SR Latch due to different path delays! To be discussed on next slide…

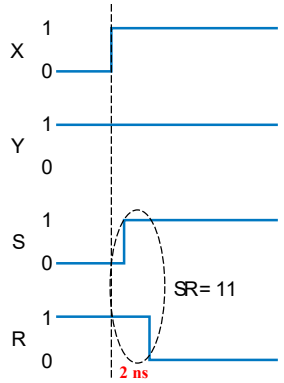
Problem with SR Latch — 2: Glitch ➔ Oscillation

- Problem occurs even if SR inputs come from a circuit that supposedly never sets S=1 and R=1 at same time
  - But 11 can occur due to different path delays



In a situation when R is already 1 (SR=01) then X becomes 1 (SR=10):

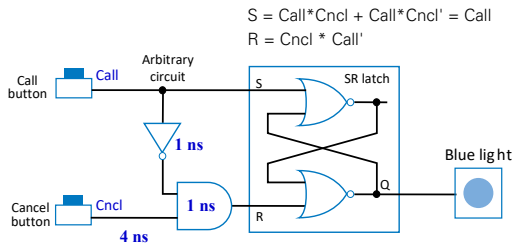
- The longer path from X to R than X to S causes SR=11 for short time – (glitch) could be long enough to **cause oscillation**

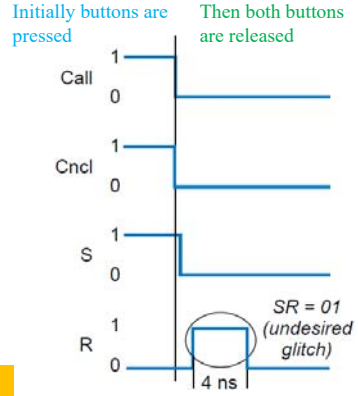


A temporary unintended signal value caused by circuit delays is called a ***glitch***.

Problem with SR Latch — 3: Glitch ➔ Set/Reset

- Assume cancel button to the AND gate wire delay is **4 ns** (perhaps the wire is very long), and **1 ns** AND and NOT gate delays.
- Suppose *both buttons are pressed (11)*, so SR=10, then *both buttons are released (00)* -- SR should become 00.
- In this situation, the longer delay from cancel button to AND gate (than call button to AND gate) causes SR=01 for short time – a **glitch** – **that could be long enough to cause reset**

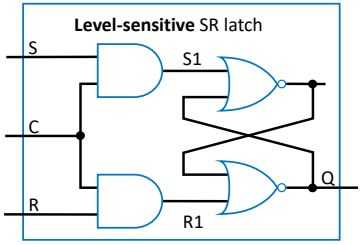




A temporary unintended signal value caused by circuit delays is called a ***glitch***.

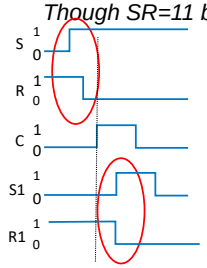
Solution to Glitches: Level-Sensitive SR Latch

- Add enable input "C" as shown
- Only let S and R change when C=0
  - Ensure circuit in front of SR never sets SR=11, except briefly due to path delays
  - Change C to 1 only after sufficient time for S and R to be stable
  - When C becomes 1, the stable S and R value passes through the two AND gates to the SR latch's S1 R1 inputs.

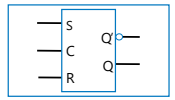


Allowing only stable SR signals avoids glitches

Though SR=11 briefly...



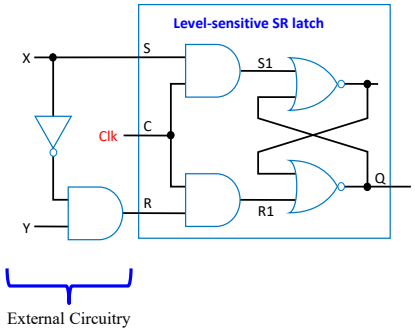
...S1R1 never = 11



Level-sensitive SR latch symbol

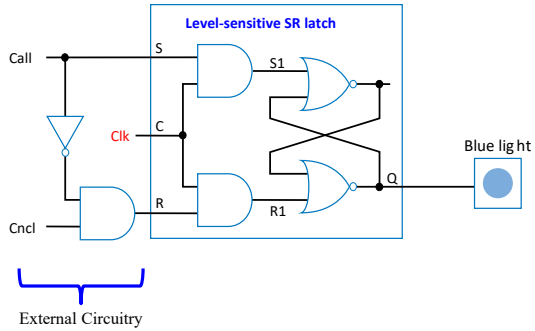
Solution to Glitches: Level-Sensitive SR Latch

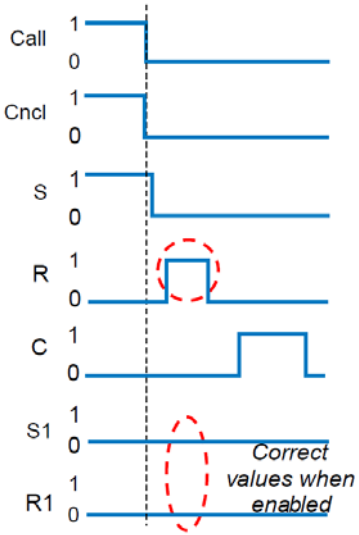
- REVISIT:
  - Problem with SR Latch — 2: Glitch ➔ Oscillation
  - Problem with SR Latch — 3: Glitch ➔ Set/Reset



Now S1R1=11 due to different path delays **can't occur** on Basic SR Latch!

Solution to Glitches: Level-Sensitive SR Latch

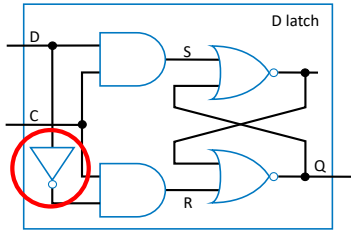


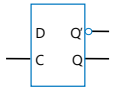


Glitch on R (or S) doesn't affect R1 (or S1)

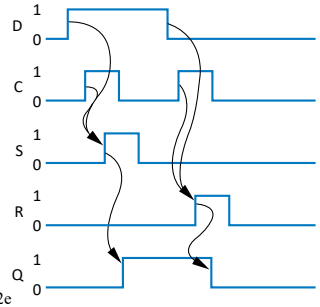
Level-Sensitive D Latch

- SR latch requires careful design to ensure SR=11 never occurs
- D latch relieves designer of that burden
  - Inserted inverter ensures R always opposite of S



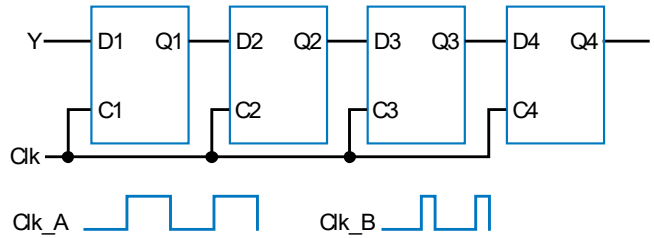


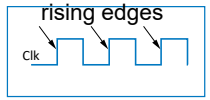
D latch symbol



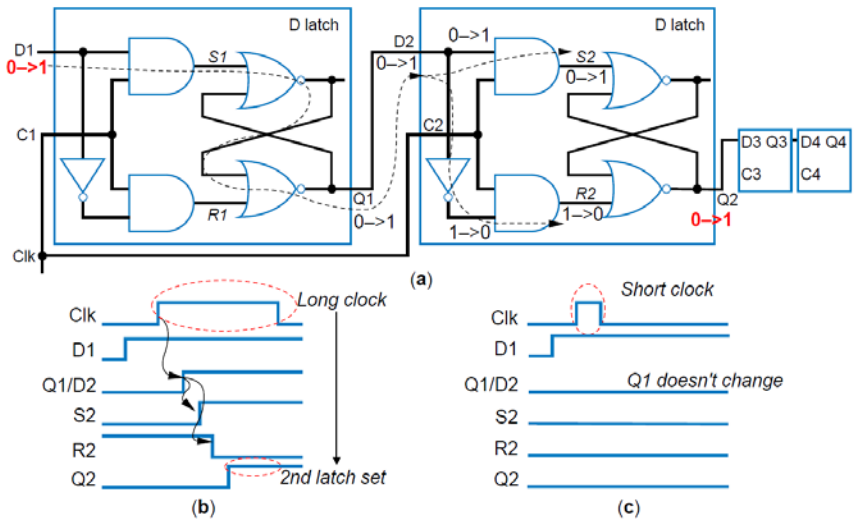
Problem with Level-Sensitive D Latch

- D latch still has problem (as does SR latch)
  - When C=1, through how many latches will a signal travel?
  - Depends on for how long C=1
    - Clk_A -- signal may travel through multiple latches
    - Clk_B -- signal may travel through fewer latches
  - Hard to pick C that is just the right length
    - Can we design bit storage that only stores a value on the rising edge of a clock signal?





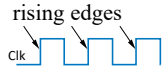
Problem with Level-Sensitive D Latch



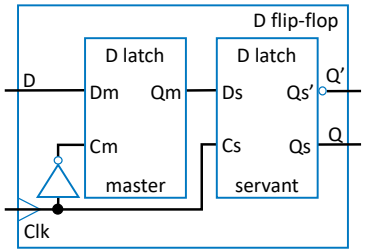
D Flip-Flop

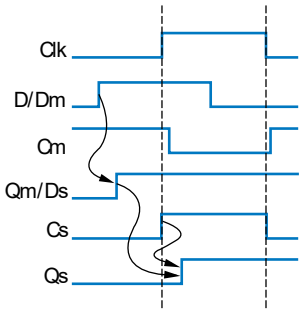
Can we design bit storage that only stores a value on the *rising* edge of a clock signal?

- ***Flip-flop***: Bit storage that stores on clock **edge**, not level
- One design -- master-servant
  - Two latches, output of first goes to input of second, master latch has inverted clock signal
    - Master loaded with D when Clk=0, then servant when Clk=1
    - When Clk changes from 0 to 1, master disabled, servant loaded with value that was at D just before Clk changed
    - Thus, D flip-flop loads value at D during rising edge of Clk



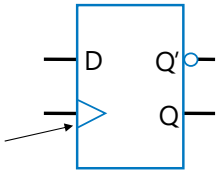
Note: Hundreds of different flip-flop designs exist



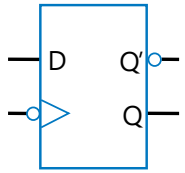


D Flip-Flop

The **triangle** means clock input, edge triggered

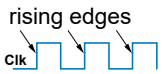


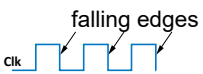
Symbol for rising-edge triggered D flip-flop



Symbol for falling-edge triggered D flip-flop

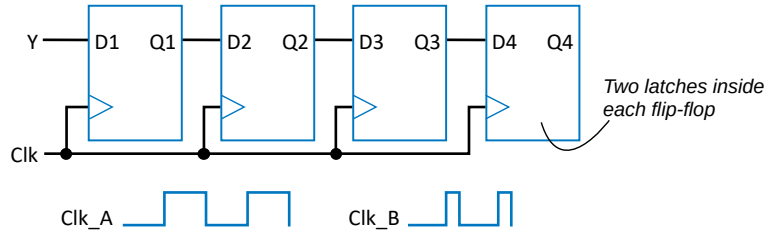
Internal design: Just invert servant clock rather than master





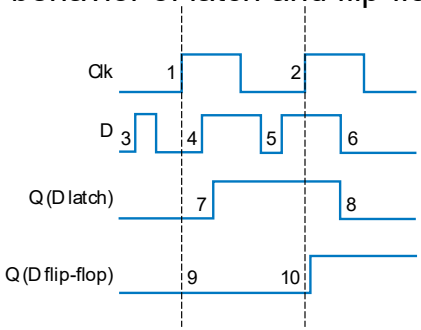
D Flip-Flop

- Solves problem of not knowing through how many latches a signal travels when C=1
  - In figure below, signal travels through exactly one flip-flop, for Clk_A or Clk_B
  - Why? Because on rising edge of Clk, all four flip-flops are loaded simultaneously -- then all four no longer pay attention to their input, until the next rising edge. Doesn't matter how long Clk is 1.



D Latch vs. D Flip-Flop

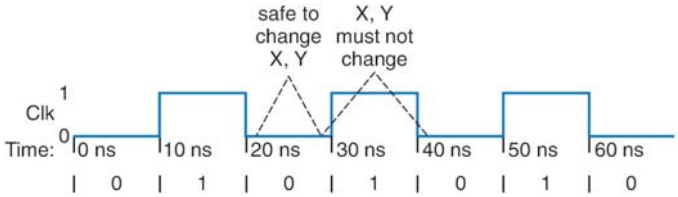
- **Latch** is level-sensitive: Stores D when C=1
- **Flip-flop** is edge triggered: Stores D when C changes from 0 to 1
  - Saying "level-sensitive latch," or "edge-triggered flip-flop," is redundant
  - Two types of flip-flops -- rising or falling edge triggered.
- Comparing behavior of latch and flip-flop:



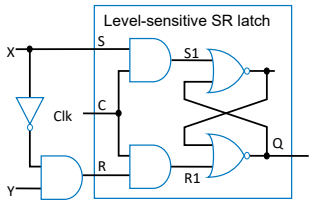
Latch follows D while Clk is 1

Flip-flop only loads D during Clk rising edge

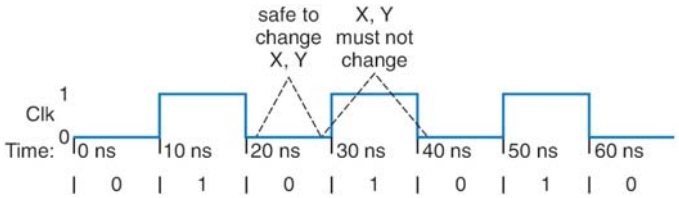
Clock Signals for a Latch



- How do we know when it's safe to set C=1?
  - Most common solution –make C pulse up/down
    - C=0: Safe to change X, Y
    - C=1: Must *not* change X, Y
    - We'll see how to ensure that later
  - ***Clock*** signal -- Pulsing signal used to enable latches
    - Because it ticks like a clock
  - Sequential circuit whose storage components all use clock signals: ***synchronous*** circuit
    - Most common type
    - Asynchronous circuits – important topic, but left for advanced course



Clocks

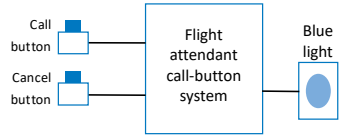


- ***Clock period***: time interval between pulses
  - Above signal: period = 20 ns
- ***Clock cycle***: one such time interval
  - Above signal shows 3.5 clock cycles
- ***Clock frequency***: 1/period
  - Above signal: frequency = 1 / 20 ns = 50 MHz
    - 1 Hz = 1/s

Freq	Period
100 GHz	0.01 ns
10 GHz	0.1 ns
1 GHz	**1 ns**
100 MHz	10 ns
10 MHz	100 ns

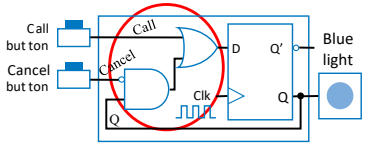
Flight-Attendant Call Button Using D Flip-Flop

- D flip-flop will store bit
- Inputs are Call, Cancel, and present value of D flip-flop, Q
- Truth table shown below



Call	Cancel	Q	D	
0	0	0	0	Preserve value: if Q=0, make D=0; if Q=1, make D=1
0	0	1	1	
0	1	0	0	Cancel -- make D=0
0	1	1	0	
1	0	0	1	Call -- make D=1
1	0	1	1	
1	1	0	1	Let's give **priority to Call** -- make D=1
1	1	1	1	

Encircled circuit derived from truth table, using Chapter 2 combinational logic design process

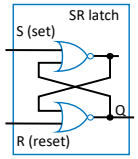


This circuit is more robust than the earlier circuit using an SR latch (shown earlier).

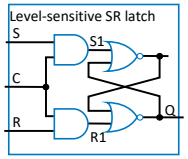
However, it still needs some improvements… as we see later!

$$D = Cancel' \, Q + Call$$

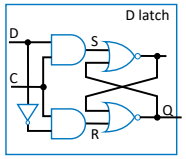
Bit Storage Summary



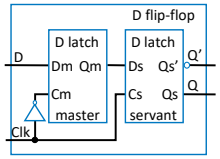
Feature: S=1 sets Q to 1, R=1 resets Q to 0. Problem: SR=11 yield undefined Q.



Feature: S and R only have effect when C=1. We can design outside circuit so SR=11 never happens when C=1. Problem: avoiding SR=11 can be a burden.



Feature: SR can't be 11 if D is stable before and while C=1, and will be 11 for only a brief glitch even if D changes while C=1. Problem: C=1 too long propagates new values through too many latches: too short may not enable a store.



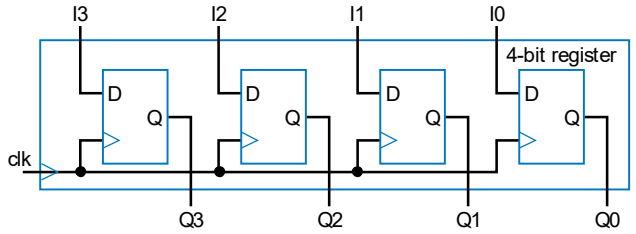
Feature: Only loads D value present at rising clock edge, so values can't propagate to other flip-flops during same clock cycle. Tradeoff: uses more gates internally than D latch, and requires more external gates than SR – but gate count is less of an issue today.

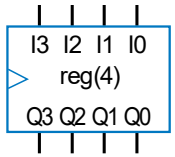
- We considered increasingly better bit storage until we arrived at the robust D flip-flop bit storage

Digital Design 2e
Copyright © 2010
Frank Vahid

Basic Register

- Typically, we store multi-bit items
  - e.g., storing a 4-bit binary number
- ***Register***: multiple flip-flops sharing clock signal
  - From this point, we'll use registers for bit storage
    - No need to think of latches or flip-flops
    - But now you know what's inside a register

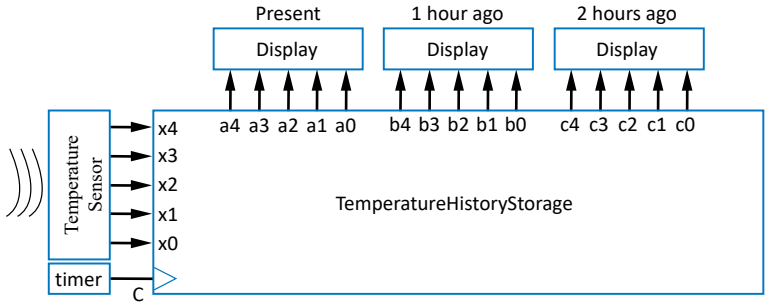




Digital Design 2e
Copyright © 2010
Frank Vahid

Example Using Registers: Temperature Display

- Temperature history display
  - Sensor outputs temperature as 5-bit binary number
  - Timer pulses C every hour
  - Record temperature on each pulse, display last three recorded values

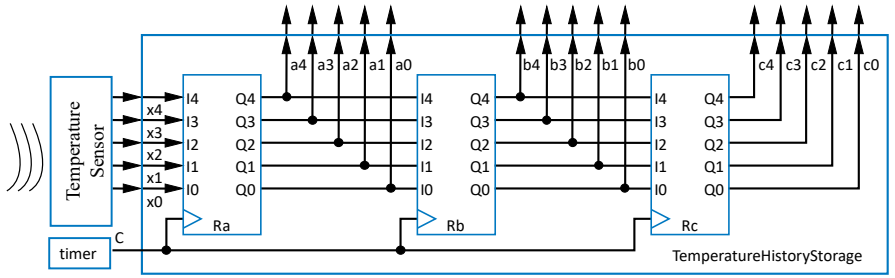


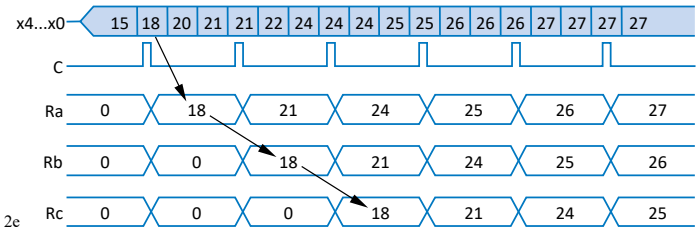
(In practice, we would actually avoid connecting the timer output C to a clock input, instead only connecting an oscillator output to a clock input.)

Digital Design 2e
Copyright © 2010
Frank Vahid

Example Using Registers: Temperature Display

- Use three 5-bit registers

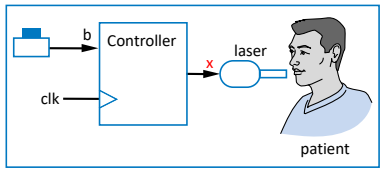




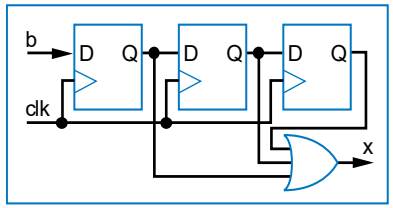
Digital Design 2e
Copyright © 2010
Frank Vahid

Finite-State Machines (FSMs) and Controllers

- Want sequential circuit with particular behavior over time
  - **State** of the circuit: State can be intentionally used to design circuits that have a specific behavior over time.
- **Example: Laser timer**
  - Push button b causes x=1 for 3 clock cycles
  - How? Let's try three flip-flops
    - b=1 gets stored in first D flip-flop
    - Then 2nd flip-flop on next cycle, then 3rd flip-flop on next
    - OR the three flip-flop outputs, so x should be 1 for three cycles



A circuit's *state* is the value of all a circuit's stored bits.



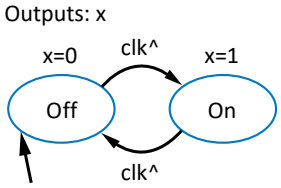
Three-cycles-high laser timer - a poorly done first design – what if button pressed a second time during those 3 cycles?

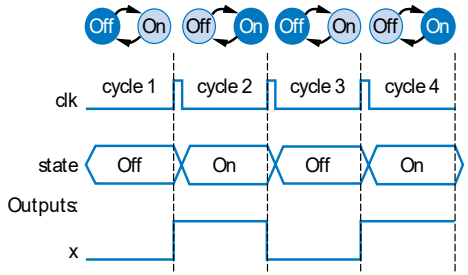
Need a Better Way to Design Sequential Circuits

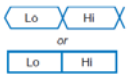
- **Trial and error** is not a good design method
  - Will we be able to "guess" a circuit that works for other desired behavior?
    - How about counting up from 1 to 9? Pulsing an output for 1 cycle every 10 cycles? Detecting the sequence 1 3 5 in binary on a 3-bit input?
  - And, a circuit built by guessing may have undesired behavior
    - Laser timer: What if press button again while x=1? x then stays one another 3 cycles. Is that what we want?
- **Combinational circuit design** process had two important things
  1. Capture: A formal way to describe desired circuit behavior
     - Boolean equation, or truth table
  2. Convert: A well-defined process to convert that behavior to a circuit
- **We need those things for sequence circuit design**

Describing Behavior of Sequential Circuit: FSM

- Finite-State Machine (FSM)
  - A way to describe desired behavior of sequential circuit
    - Akin to Boolean equations for combinational behavior
- List all possible states, and transitions among states
  - Example: Make x change (0 to 1, or 1 to 0) every clock cycle i.e., toggle
  - Two states: "Off" (x=0), and "On" (x=1)
  - Transition from Off to On, or On to Off, on rising clock edge
  - Arrow with no starting state points to initial state (when circuit first starts)







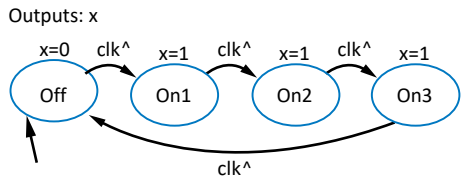
While representing multibit data, a vertical line (or a pair of crossed lines) shows when the values change.

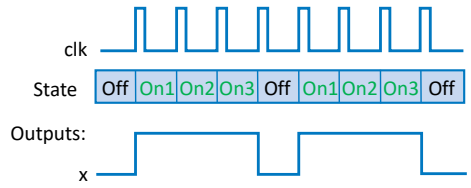
FSM Example: 0,1,1,1,repeat

Example: Build a system that held its output high for three cycles, return to 0 and repeats

- **Want 0, 1, 1, 1, 0, 1, 1, 1, ...**
  - Each value for one clock cycle
- Capture as FSM
  - Four states
  - Transition on rising clock edge to next state

The state names in state machine could have been *S0*, *S1*, *S2*, and *S3*, but names that describe a state's purpose in the FSM are preferable.

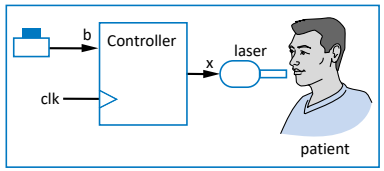


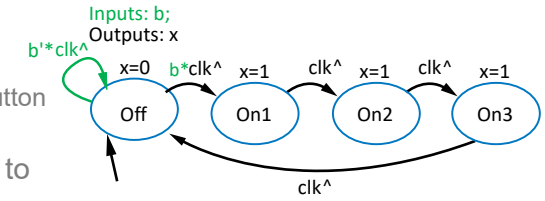


NEXT ➔ Extend this FSM to Three-Cycles High Laser Timer

Extend FSM to Three-Cycles High Laser Timer

- Four states
- Wait in "Off" state while b is 0 (b′)
- When b is 1 (and rising clock edge), transition to On1
  - Sets x=1
- On next two clock edges, transition to On2, then On3
  - also set x=1 in each state
  - So x=1 for three cycles after button pressed
- On next clock edge, transition to Off
  - Sets x=0

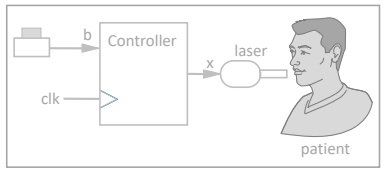


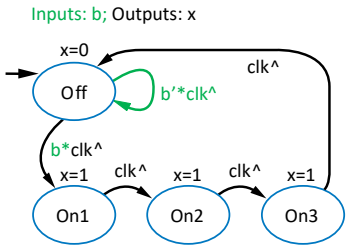


Boolean expressions can be associated with the transitions to extend the behavior

Extend FSM to Three-Cycles High Laser Timer

- Four states
- Wait in "Off" state while b is 0 (b′)
- When b is 1 (and rising clock edge), transition to On1
  - Sets x=1
- On next two clock edges, transition to On2, then On3
  - also set x=1 in each state
  - So x=1 for three cycles after button pressed
- On next clock edge, transition to Off
  - Sets x=0

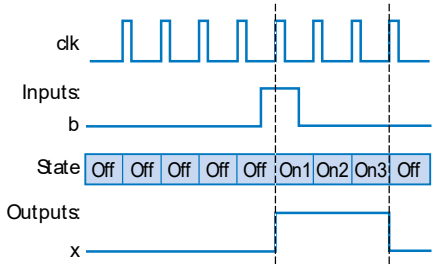


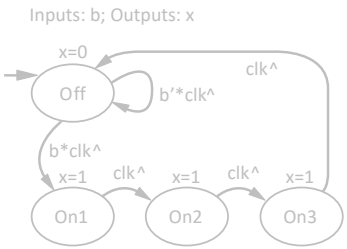


Boolean expressions can be associated with the transitions to extend the behavior

Extend FSM to Three-Cycles High Laser Timer

- Four states
- Wait in "Off" state while b is 0 (b′)
- When b is 1 (and rising clock edge), transition to On1
  - Sets x=1
- On next two clock edges, transition to On2, then On3
  - also set x=1 in each state
  - So x=1 for three cycles after button pressed
- On next clock edge, transition to Off
  - Sets x=0

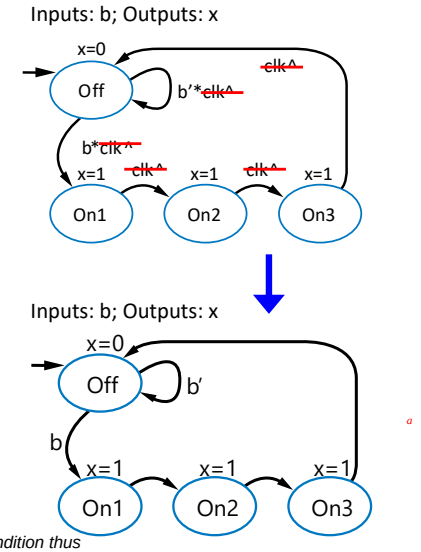




Boolean expressions can be associated with the transitions to extend the behavior

FSM Simplification: Rising Clock Edges Implicit

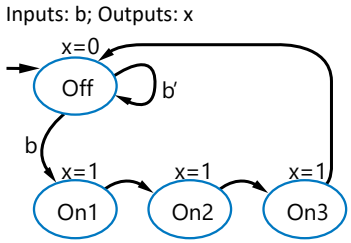
- Showing rising clock on every transition: cluttered
  - Make **implicit** -- assume every edge has rising clock, even if not shown
  - What if we wanted a transition *without* a rising edge
    - We don't consider such asynchronous FSMs -- less common, and advanced topic
    - Only consider ***synchronous*** FSMs -- rising edge on *every* transition



Note: Transition with no associated condition thus transistions to next state on next clock cycle

FSM Definition

- **FSM consists of**
  - Set of states
    - Ex: {Off, On1, On2, On3}
  - Set of inputs, set of outputs
    - Ex: Inputs: {b}, Outputs: {x}
  - Initial state
    - Ex: "Off"
  - Set of transitions
    - Directed edges with associated input conditions to indicate next states
    - Ex: Has 5 transitions
  - Set of actions
    - Sets outputs while in states
    - Ex: x=0, x=1, x=1, and x=1



We often draw FSM graphically, known as ***state diagram***

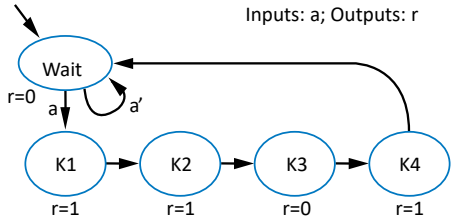
Can also use table (state table), or textual languages

Assigning an output in an FSM is known as an *action.*

FSM Example: Secure Car Key

- Many new car keys include tiny computer chip
  - When car starts, car's computer (under engine hood) requests identifier from key
  - Key transmits identifier
    - If not, computer shuts off car
- FSM for chip in the key
  - Wait until computer requests ID (a=1)
  - Transmit ID
    - in this exemplary case, 1101

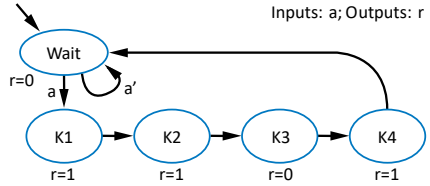




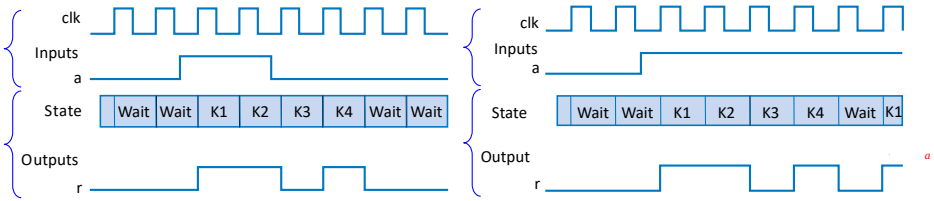
LSB ➔ MSB

FSM Example: Secure Car Key (cont.)

- Nice feature of FSM
  - Executing or Running an FSM
    - Can evaluate output behavior for different input sequence
  - Timing diagrams represents
    - represent a particular situation defined by how we set the inputs.
    - show states and output values for different input waveforms

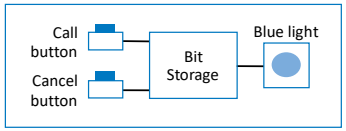


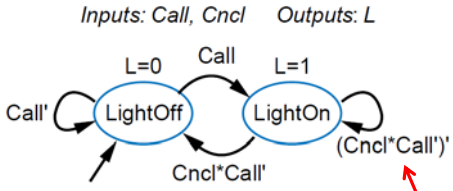
Q: Determine states and r value for given input waveform:

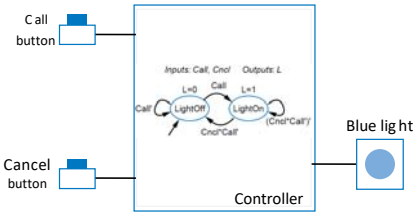


Flight-attendant call button

- Previously built using SR latch, then D flip-flop
- Capture using FSM instead







Only condition to transition back
F(Cncl, Call) = Cncl * Call'
= m2
1,0

Rest of the conditions for no change
F'(Cncl, Call) = (Cncl * Call')'
= Cncl' + Call
= m0 + m1 + m3
0,0
0,1
1,1

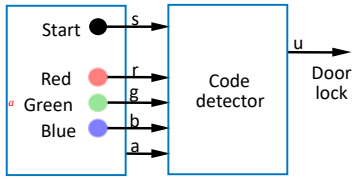
More explicit transitions first: The only way to transition back to *Ligh10ff* is if encl is 1 and Call is 0.

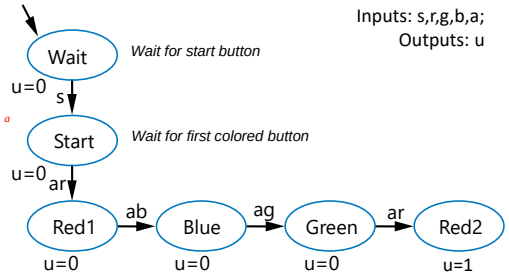
Capturing Desired System Behavior as an FSM

- A challenging task for a designer
- Following method can help
  - List States
  - Create transitions
  - Refine the FSM
- The method described above is just a guide.
- Capturing behavior as an FSM may require some creativity and trial-and-error
- Many different FSMs could be created that describe the same desired behavior
  - one FSM may be easier to understand while another FSM may have fewer states
- Experience can help greatly in creating correct and easy-to-understand FSMs that capture desired system behavior

FSM Example: Code Detector

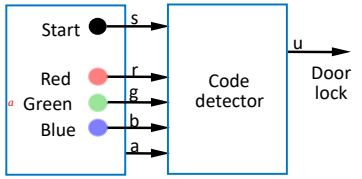
- Unlock door (u=1) only when buttons pressed in sequence:
  - First **start**, then **red**, **blue**, **green**, **red**
- Input from each button: *s, r, g, b*
  - Also, output *a* from keypad indicates that some colored button pressed
- FSM
  - Wait for start (s=1) in "Wait"
  - Once started ("Start")
    - If see red, go to "Red1"
    - Then, if see blue, go to "Blue"
    - Then, if see green, go to "Green"
    - Then, if see red, go to "Red2"
      - In that state, open the door (u=1)
    - Wrong button at any step, return to "Wait", without opening door

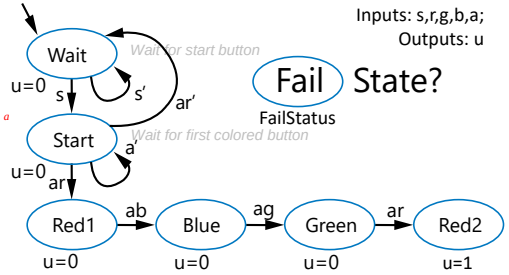




FSM Example: Code Detector

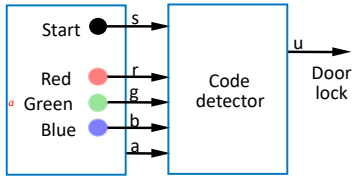
- Unlock door (u=1) only when buttons pressed in sequence:
  - First **start**, then **red**, **blue**, **green**, **red**
- Input from each button: *s, r, g, b*
  - Also, output *a* from keypad indicates that some colored button pressed
- FSM
  - Wait for start (s=1) in "Wait"
  - Once started ("Start")
    - If see red, go to "Red1"
    - Then, if see blue, go to "Blue"
    - Then, if see green, go to "Green"
    - Then, if see red, go to "Red2"
      - In that state, open the door (u=1)
    - Wrong button at any step, return to "Wait", without opening door

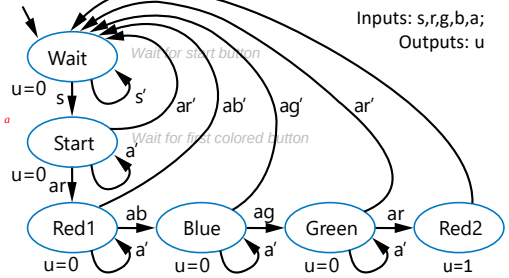




FSM Example: Code Detector

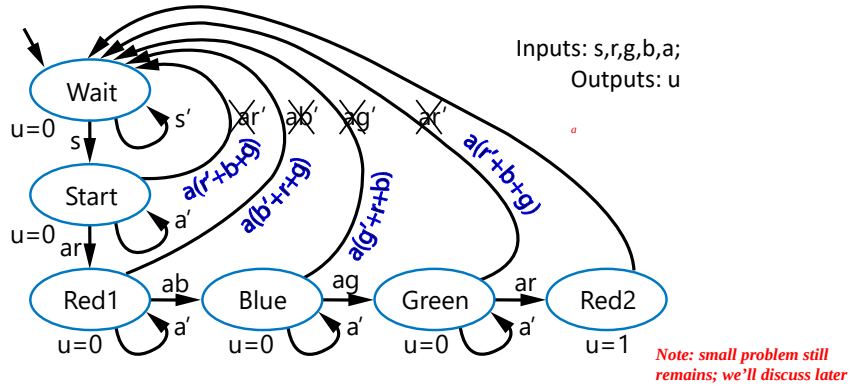
- Unlock door (u=1) only when buttons pressed in sequence:
  - First **start**, then **red**, **blue**, **green**, **red**
- Input from each button: *s, r, g, b*
  - Also, output *a* from keypad indicates that some colored button pressed
- FSM
  - Wait for start (s=1) in "Wait"
  - Once started ("Start")
    - If see red, go to "Red1"
    - Then, if see blue, go to "Blue"
    - Then, if see green, go to "Green"
    - Then, if see red, go to "Red2"
      - In that state, open the door (u=1)
    - Wrong button at any step, return to "Wait", without opening door





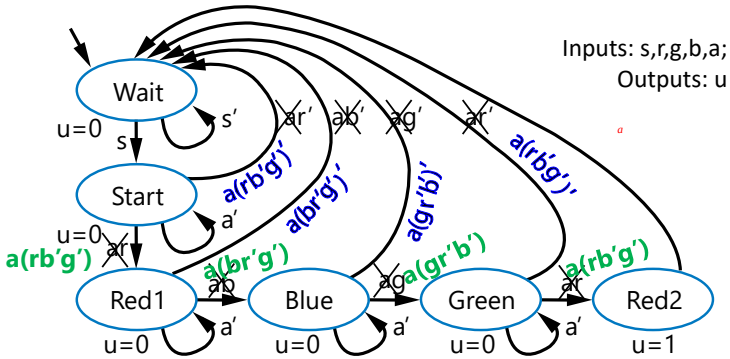
What happens if the user presses the start button and then presses *all three colored buttons simultaneously,* four times in a row? The way the FSM is defined, the door would unlock!

Improve FSM for Code Detector (cont.)



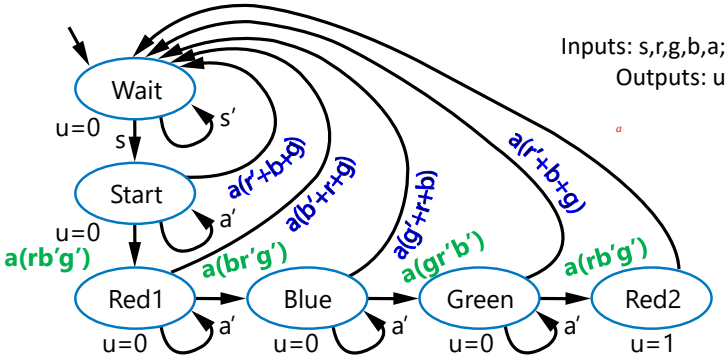
- New transition conditions detect if wrong button pressed, returns to "Wait"
- FSM provides formal, concrete means to accurately define desired behavior

Improve FSM for Code Detector (cont.)



- New transition conditions enables only one transition
- New transition conditions detect if wrong button pressed, returns to "Wait"
- FSM provides formal, concrete means to accurately define desired behavior

Improve FSM for Code Detector (cont.)



- New transition conditions enables only one transition
- New transition conditions detect if wrong button pressed, returns to "Wait"
- FSM provides formal, concrete means to accurately define desired behavior